

Pattern Recognition and Text Processing

Course Code: DSA03 Course Title: Natural Language Processing
 CO Assessed: CO1 – Imitation (L1) Assessment: Assessment Tool 1 – Pattern Recognition and Text Processing
 Weightage: 40% of CIA
 CO5 Statement: Analyse regular expression patterns. (BL-3)
 Student Name: G. Viraj Kumar Reddy Reg. No.: 192472093
 Section / Batch: CSE-A1 Date:

Code of Conduct

I, G. Viraj Kumar Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Viraj

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

85

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

SMARTS ENGINEERING

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

NAME: G. Vinay Kumar Reddy

REG NO: 192472093

COURSE CODE: DSA0306

COURSE NAME: NATURAL LANGUAGE PROCESSING

1) Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

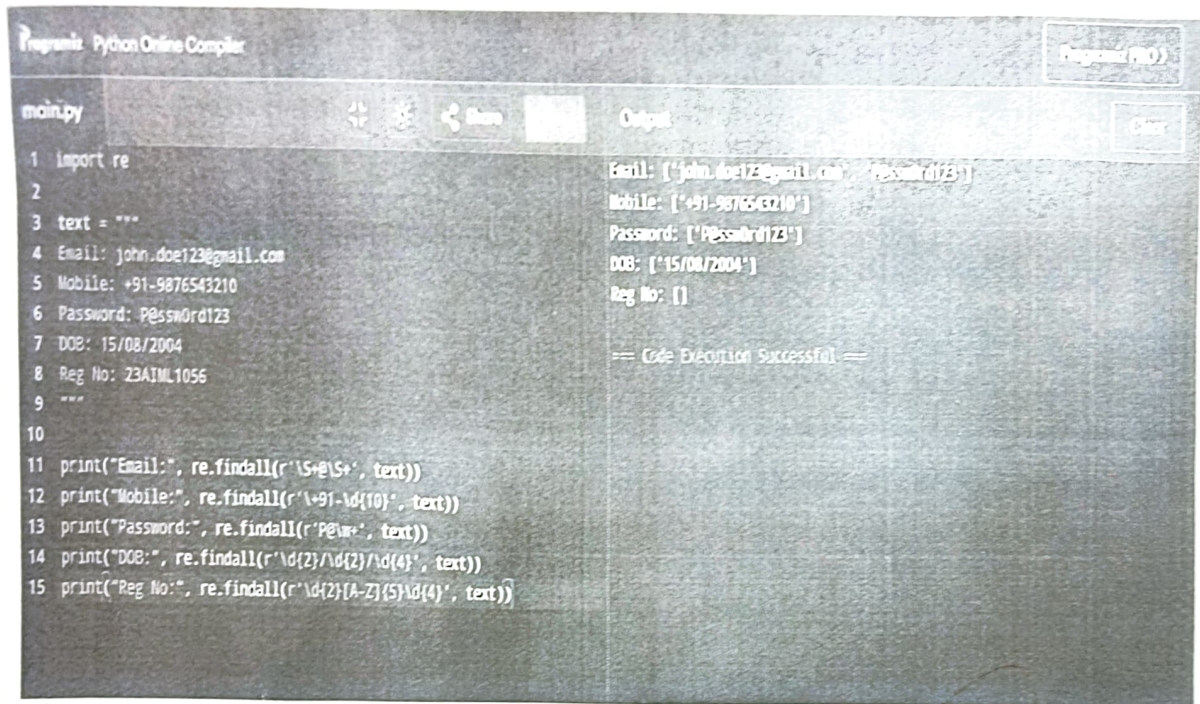
Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number



```
Programs Python Online Compiler
main.py
1 import re
2
3 text = """
4 Email: john.doe123@gmail.com
5 Mobile: +91-9876543210
6 Password: P@ssw0rd123
7 DOB: 15/08/2004
8 Reg No: 23A1ML1056
9 """
10
11 print("Email:", re.findall(r'\S+@(\S+)', text))
12 print("Mobile:", re.findall(r'\+91-\d{10}', text))
13 print("Password:", re.findall(r'P@w*', text))
14 print("DOB:", re.findall(r'\d{2}/\d{2}/\d{4}', text))
15 print("Reg No:", re.findall(r'\d{2}[A-Z]{5}\d{4}', text))

Output
Email: ['john.doe123@gmail.com', 'P@ssw0rd123']
Mobile: ['+91-9876543210']
Password: ['P@ssw0rd123']
DOB: ['15/08/2004']
Reg No: []

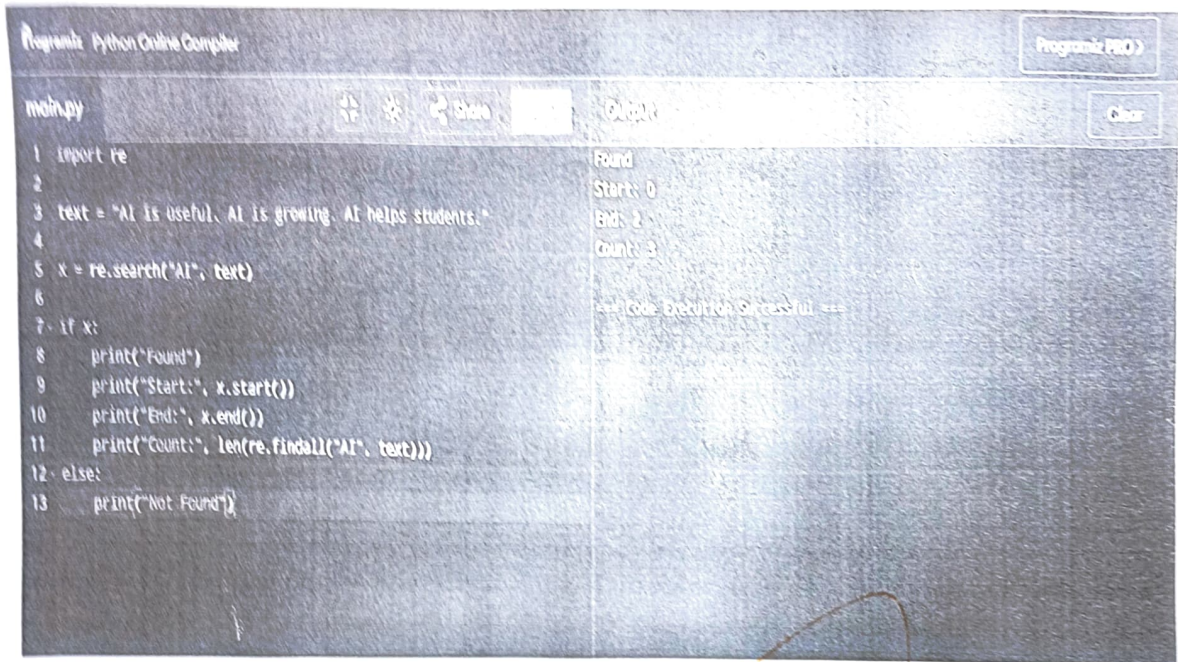
== Code Execution Successful ==
```

2) Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.



The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

```
1 import re
2
3 text = "AI is useful. AI is growing. AI helps students."
4
5 x = re.search("AI", text)
6
7 if x:
8     print("Found")
9     print("Start:", x.start())
10    print("End:", x.end())
11    print("Count:", len(re.findall("AI", text)))
12 else:
13    print("Not Found")
```

The output on the right side of the compiler shows the following results:

```
Found
Start: 0
End: 2
Count: 3

=== Code Execution Successful ===
```

3) Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the `re.split()` method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

The screenshot shows a web-based Python IDE. The editor on the left contains a Python script that uses the `re` module to split a text string into sentences and words. The output panel on the right displays the results of the script's execution.

```
1 import re
2
3 text = "AI is useful. AI helps students! AI is growing?"
4 sentences = re.split(r'[.!?]', text)
5 sentences = [s for s in sentences if s != ""]
6
7
8 words = re.split(r'\s+', text)
9
10 print("Sentences:", sentences)
11 print("Sentence Count:", len(sentences))
12
13 print("Words:", words)
14 print("Word Count:", len(words))
```

Output:

```
Sentences: ['AI is useful', ' AI helps students!', ' AI is growing?']
Sentence Count: 3
Words: ['AI', 'is', 'useful.', 'AI', 'helps', 'students!', 'AI', 'is', 'growing?']
Word Count: 9

=== Code Execution Successful ===
```

4) Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the `re` module to validate the following inputs.

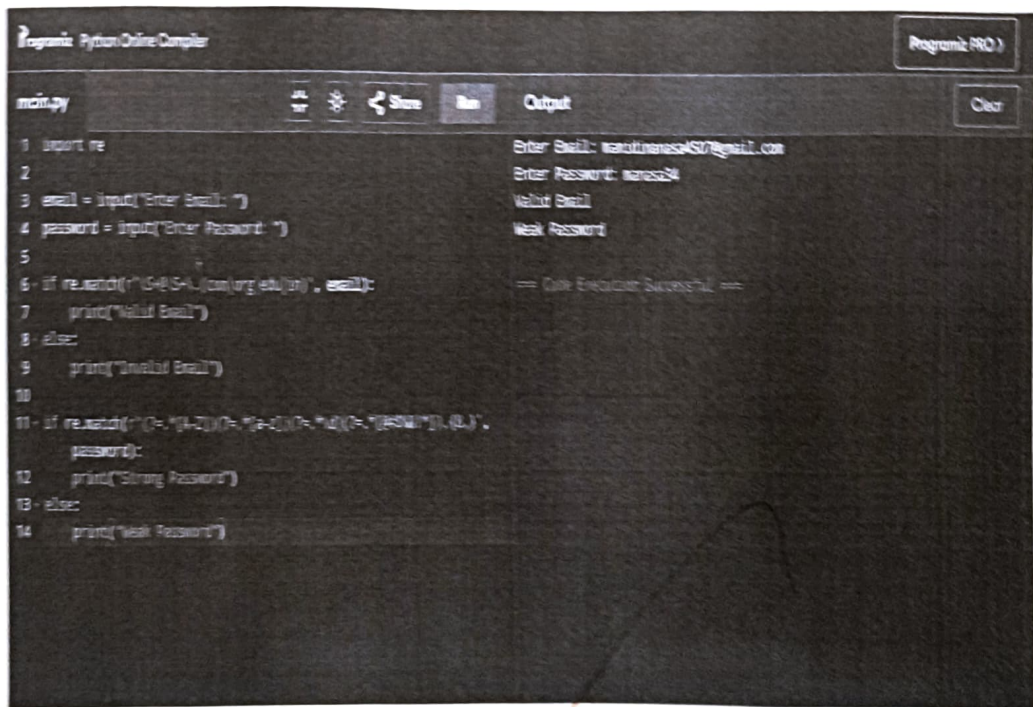
Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain `.`, `_`, `%`, `+`, or `-` before the `@` symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as `.com`, `.org`, `.edu`, or `.in`.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.



The screenshot shows a Python Online Compiler interface. The code defines a function `isStrongPassword` that checks if a password is strong based on the criteria listed above. The program prompts the user to enter an email and a password, then prints the result of the `isStrongPassword` function.

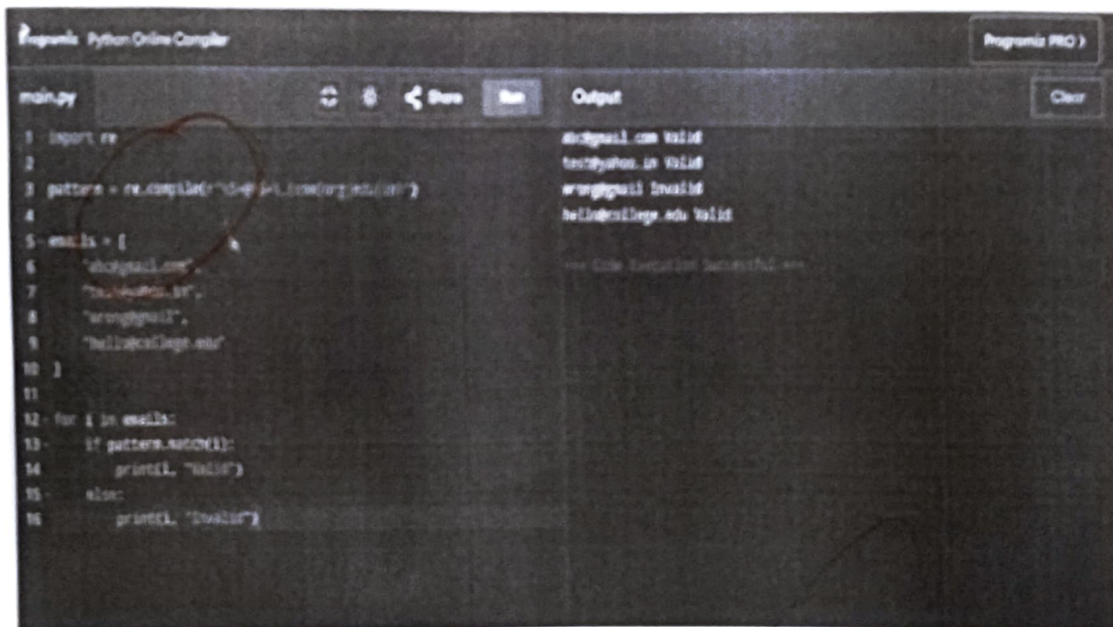
```
1 import re
2
3 email = input("Enter Email: ")
4 password = input("Enter Password: ")
5
6 if re.match(r"^[a-zA-Z0-9@org.edu]{1,}$", email):
7     print("Valid Email")
8 else:
9     print("Invalid Email")
10
11 if re.match(r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[8,]$", password):
12     print("Strong Password")
13 else:
14     print("Weak Password")
```

Output:

```
Enter Email: manojnasa450@gmail.com
Enter Password: manasa450
Valid Email
Weak Password
== Code Execution Successful ==
```

5) Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the `re.compile()` method to compile the pattern once and reuse it to validate all the email IDs.



```
main.py
1 import re
2
3 pattern = re.compile(r"[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,4}")
4
5 emails = [
6     'abc@gmail.com',
7     'test@python.in',
8     'wrong@.com',
9     'hello@college.edu'
10 ]
11
12 for i in emails:
13     if pattern.match(i):
14         print(i, "Valid")
15     else:
16         print(i, "Invalid")
```

Program (PID: ...)

Output

```
abc@gmail.com Valid
test@python.in Valid
wrong@.com Invalid
hello@college.edu Valid
```

*** Code Execution Successful ***

Rubrics

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1						15	75
2						15	
3						15	
4						15	
5						15	

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____